

Aegis Update Authority Check

Adding Metadata Authority Detection to the Existing Risk Engine

Feature Extension | June 2026

Context: What Already Exists

Aegis already detects and scores Mint Authority and Freeze Authority as part of the token contract scan. This document adds **Update Authority** as a third authority check — plugged directly into the existing risk scoring engine without changing the current architecture.

Authority	Status	Current Score Impact
Mint Authority	Already Implemented ■	Active: +30 pts (Danger)
Freeze Authority	Already Implemented ■	Active: +30 pts (Danger)
Update Authority	NEW — Adding Now ■	Active: +15 pts (Warning)

Why Update Authority Matters

Update Authority controls the token's on-chain metadata — specifically its name, symbol, image, and description. Unlike Mint and Freeze Authority, it doesn't affect token supply or transfers directly, but it enables a specific class of attack: **metadata manipulation after gaining trust**.

Attack scenario: Creator launches "\$LEGIT" with a real logo and description. Token gains traction and liquidity. Creator quietly renames it "\$SCAM", changes the image to something malicious, and exits. Holders are left with a rebranded worthless token.

Scoring Rationale: Why +15 Points (Warning)

Authority	Score Impact	Severity	Reason
Mint Authority	+30 (Danger)	Critical	Can inflate supply and crash price
Freeze Authority	+30 (Danger)	Critical	Can trap user funds immediately
Update Authority	+15 (Warning)	Medium	Metadata risk; no direct fund loss

Step 1: Fetch Update Authority from Metaplex

Add this utility function to your existing token scanning module. It uses Metaplex metadata to retrieve the update authority address.

```
// Add to your existing token scanner lib
import { Connection, PublicKey } from '@solana/web3.js';
import { Metaplex } from '@metaplex-foundation/js';

export async function getUpdateAuthority(
  mintAddress: string,
  connection: Connection // reuse your existing Helius connection
): Promise<string | null> {
  try {
    const metaplex = Metaplex.make(connection);
    const mint = new PublicKey(mintAddress);

    const nft = await metaplex.nfts().findByMint({ mintAddress: mint });
    return nft.updateAuthorityAddress?.toString() || null;
  } catch {
    // Token may have no metadata (valid case)
    return null;
  }
}
```

Step 2: Plug Into Your Existing Risk Scoring Engine

Your current engine already calculates scores from mint and freeze authority. Add update authority as a third modifier using the same pattern:

```
// Your existing scoring function – add the highlighted block

export async function calculateTokenRiskScore(mintAddress: string): Promise<RiskScore> {
  let score = 0;
  const signals: string[] = [];

  // --- EXISTING: Mint Authority ---
  if (mintAuthority && mintAuthority !== 'None') {
    score += 30;
    signals.push({ label: 'Active Mint Authority', severity: 'danger' });
  }

  // --- EXISTING: Freeze Authority ---
  if (freezeAuthority && freezeAuthority !== 'None') {
    score += 30;
    signals.push({ label: 'Active Freeze Authority', severity: 'danger' });
  }

  // ■ NEW: Update Authority – add these lines below the existing checks
  const updateAuthority = await getUpdateAuthority(mintAddress, connection);
  if (updateAuthority && updateAuthority !== 'None') {
    score += 15;
    signals.push({
      label: 'Active Update Authority',
      severity: 'warning',
      detail: 'Token metadata can be changed by the creator'
    });
  }
}
```

```

}

// --- EXISTING: Renounced Authorities ---
const allRevoked =
  (!mintAuthority || mintAuthority === 'None') &&
  (!freezeAuthority || freezeAuthority === 'None') &&
  (!updateAuthority || updateAuthority === 'None'); // ■ include update auth here

if (allRevoked) {
  score -= 10;
  signals.push({ label: 'All Authorities Renounced', severity: 'info' });
}

return { score: Math.max(0, score), signals, riskLevel: classifyScore(score) };
}

```

Step 3: Surface It in the Scanner UI

Update your token results component to display Update Authority alongside the existing Mint and Freeze authority badges:

```
// In your token scan results component – add the third authority row

{/* EXISTING */}
<AuthorityRow
  label="Mint Authority"
  active={result.mintAuthority !== null}
  severity="danger"
/>

{/* EXISTING */}
<AuthorityRow
  label="Freeze Authority"
  active={result.freezeAuthority !== null}
  severity="danger"
/>

{/* ■ NEW – add this */}
<AuthorityRow
  label="Update Authority"
  active={result.updateAuthority !== null}
  severity="warning"
  tooltip="Creator can modify token name, symbol, and image"
/>
```

Step 4: Reuse Your Existing AuthorityRow Component

If you already have an AuthorityRow or badge component for Mint and Freeze, Update Authority reuses the same component — just pass severity='warning' instead of 'danger':

```
// AuthorityRow component – should already handle this with severity prop
// severity='danger' → red badge (Mint, Freeze)
// severity='warning' → yellow badge (Update) ← NEW usage

// If severity prop doesn't exist yet, add it:
interface AuthorityRowProps {
  label: string;
  active: boolean;
  severity: 'danger' | 'warning' | 'info'; // add 'warning' if not present
  tooltip?: string;
}

const severityStyles = {
  danger: { bg: '#fef2f2', text: '#dc2626', badge: 'ACTIVE – DANGER' },
  warning: { bg: '#fffbeb', text: '#d97706', badge: 'ACTIVE – WARNING' }, // ← NEW
  info: { bg: '#f0f9ff', text: '#0369a1', badge: 'REVOKED' },
};
```

Score Impact Examples

Token Scenario	Score Breakdown	Final Score	Risk Level
Pump.fun graduated (all authorities null)	Mint: 0 Freeze: 0 Update: 0 All Revoked Bonus: -10	0 (LOW)	LOW ■
Update authority only (mint + freeze revoked)	Mint: 0 Freeze: 0 Update: +15	15 (LOW)	LOW ■■
Freeze + Update active (mint revoked)	Mint: 0 Freeze: +30 Update: +15	45 (MEDIUM)	MEDIUM ■
All three active (new/unvetted token)	Mint: +30 Freeze: +30 Update: +15	75 (HIGH)	HIGH ■

Implementation Checklist

- ✓ Install dependency: @metaplex-foundation/js (if not already in project)
- ✓ Add getUpdateAuthority() utility to existing token scanner lib
- ✓ Add the +15 update authority modifier to calculateTokenRiskScore()
- ✓ Update allRevoked condition to include updateAuthority check
- ✓ Add Update Authority row to token scan results UI
- ✓ Extend AuthorityRow component to support severity='warning' styling
- ✓ Test with a Pump.fun graduated token — all three should show null
- ✓ Test with a token that has active update authority to verify +15 scoring

Dependency Note

If @metaplex-foundation/js is not yet in your package.json:

```
npm install @metaplex-foundation/js
```

If Metaplex is already being used for token metadata enrichment elsewhere in Aegis, reuse the existing Metaplex instance to avoid duplicate initialization.